

司南黑名单查询服务 (snhmd) • API 接口文档与使用手册

面向接入方（客户）技术与运维人员的对外接口说明。

版本：snhmd（司南黑名单） | 通信：HTTP + JSON | 编码：UTF-8

本服务采用统一请求信封（`appKey` / `sign` / `encryptionType` / `body`）与统一响应结构（`head` + `body`）。调用方对 `body` 业务参数按 MD5 规则加签即可。

本接口入参为个人三要素 `name`、`idCard`、`mobile`，三项均必填，缺任一项或格式非法将被直接拒绝且不计费。

查询结果为结构化对象，本服务将其序列化为 JSON 字符串置于 `body.result.range`，调用方需二次解析（见 3.1.5）。该对象包含一个黑名单总判定与 9 个风险标签。

计费提示：本服务按有效查询次数计费——查得数据（001）与查无结果（999）均计费，详见第五章。

授权提示：调用前须取得被查询人的合法授权；授权书**不通过接口传输**，由调用方经商务渠道提交我方，详见 1.5。

一、接入必读

1.1 适用范围

本文档适用于接入本平台「司南黑名单查询服务 (snhmd)」的第三方产品技术人员与日常运维人员。

1.2 接入须知

- 正式服务地址（任选其一，两域名等价）：`http://aiszcloud.cn:8080`、`http://aiszcloud.com.cn:8080`。
- 接入前需先申请开通账户，由我方分配 `appKey` 与 `appSecret`（加签密钥）。
- 本服务按有效查询次数计费（查得与查无均计费），请做好查询频次控制，避免短时间内对同一主体重复发起调用。
- 调用前须按 1.5 的要求准备并提交被查询人授权书。

1.3 接口说明

项目	说明
请求方式	POST（计费查询数查询为 GET）
通信协议	HTTP
数据格式	请求体与响应体均为 JSON
字符编码	UTF-8
超时时间	4 秒（建议客户端读超时 ≥ 10 秒）
HTTP 状态码	恒为 200；业务结果与错误均通过响应体的 <code>head.errorCode</code> / <code>body.code</code> 表达
签名	调用时需对 <code>body</code> 中的业务参数 + 我方分配的 <code>appSecret</code> 进行 MD5 加签，详见第二章

1.4 环境说明

- **正式环境**：使用正式账户，调用已开通接口，按有效查询次数计费（见第五章）。
- **测试环境**：使用测试账户，返回挡板/联调数据。
- 正式账户仅适用于正式环境，测试账户仅适用于测试环境。

1.5 授权要求

1. 调用本接口查询任何个人信息前，调用方**必须已取得被查询人合法、有效的授权**（个人信息查询授权书）。
2. 授权书**不通过 API 传输**：请求中无需、也不应携带任何授权文件、授权链接或授权类型字段，本接口的请求参数仅为 3.1.1 所列三要素。
3. 授权书由调用方**通过商务渠道提交给我方**，具体提交方式、文件格式与提交频次由商务在开通账户时与调用方约定；我方收到后进行审核、留存等后续处理。
4. 请在正式调用前与商务确认授权书提交方式并完成首次提交。

二、鉴权与加签

所有业务接口共用同一套请求信封与鉴权方式。

2.1 请求信封（顶层参数）

参数	示例	类型	必填	说明
appKey	y89snhmd	String	是	我方分配给客户的公开标识
sign	7388c24a8546864b6c6589c86e8ee816	String	是	对 body 业务参数的 MD5 签名（见 2.3）
encryptionType	1	int	否	参数加密类型，1 = 明文（默认）
body	{...}	Object	是	业务请求体，见各接口定义

注意：本服务不需要额外的产品编号参数（产品由请求路径区分）。appKey / sign / encryptionType 不参与签名计算。
调用方传入的 body 字段一律为明文（姓名 / 身份证号 / 手机号）。

2.2 鉴权校验顺序

本服务按以下顺序校验，任一失败立即返回对应 head.errorCode（参数非法直接拒绝，不计费）：

- 1. appKey 是否存在 → 否则 505001
- 2. appKey 是否匹配到账户 → 否则 505004
- 3. 账户是否有效（启用且在有效期内）→ 否则 505007
- 4. 签名是否正确 → 否则 505002

2.3 加签方式

- 1. 取出 body 中所有非空的业务参数（不含文件/字节流类型，不含值为空的参数）。
- 2. 按参数名（key）的 ASCII 升序排序；首字符相同则依次比较后续字符。
- 3. 将排序后的参数按 参数名 参数值 直接拼接，最后追加 appSecret，得到待签名串。
- 4. 对待签名串做 MD5，取 32 位小写十六进制字符串，赋值给 sign。

示例：body = { "name": "陈韞", "idCard": "440303200002163115", "mobile": "13809091009" } , appSecret = <你的密钥>。

按 ASCII 升序排序后 key 顺序为 idCard → mobile → name，待签名串为：

```
idCard440303200002163115mobile13809091009name陈韞<你的密钥>
```

sign = MD5(待签名串) 的小写十六进制值。

提示：拼接顺序由 key 的 ASCII 决定，请勿写死字段顺序。加签的范围是你实际发送的 body 全部非空字段——若额外传了本接口不需要的字段，它同样要参与加签，否则会被判 505002。

2.4 加签代码示例

Java

```
public static String sign(Map<String, String> bodyParams, String appSecret) throws Exception {
    StringBuilder sb = new StringBuilder();
    List<String> keys = new ArrayList<>(bodyParams.keySet());
    Collections.sort(keys); // ASCII 升序
    for (String k : keys) {
        String v = bodyParams.get(k);
        if (v == null || v.isEmpty()) continue; // 剔除空值
        sb.append(k).append(v);
    }
    sb.append(appSecret);
    MessageDigest md = MessageDigest.getInstance("MD5");
    byte[] digest = md.digest(sb.toString().getBytes(StandardCharsets.UTF_8));
    StringBuilder hex = new StringBuilder();
    for (byte b : digest) hex.append(String.format("%02x", b));
    return hex.toString();
}
```

Python

```
import hashlib

def sign(body_params: dict, app_secret: str) -> str:
    parts = []
    for k in sorted(body_params.keys()): # ASCII 升序
        v = body_params[k]
        if v is None or v == "":
            continue # 剔除空值
        parts.append(f"{k}{v}")
    raw = "".join(parts) + app_secret
    return hashlib.md5(raw.encode("utf-8")).hexdigest() # 小写 hex
```

Go

```
func sign(body map[string]string, appSecret string) string {
    keys := make([]string, 0, len(body))
    for k, v := range body {
        if v != "" { // 剔除空值
            keys = append(keys, k)
        }
    }
    sort.Strings(keys) // ASCII 升序
    var sb strings.Builder
    for _, k := range keys {
        sb.WriteString(k)
        sb.WriteString(body[k])
    }
    sb.WriteString(appSecret)
    sum := md5.Sum([]byte(sb.String()))
    return hex.EncodeToString(sum[:]) // 小写 hex
}
```

三、接口列表

3.1 司南黑名单查询 (snhmd)

项目	内容
路径	POST /v1/openapi/zlx/querySrmxSNHMD
完整地址	http://aiszcloud.cn:8080/v1/openapi/zlx/querySrmxSNHMD (或 http://aiszcloud.com.cn:8080/v1/openapi/zlx/querySrmxSNHMD)
鉴权	appKey + MD5 签名 (见第二章)

3.1.1 请求 body 参数

参数	示例	类型	必填	说明
name	陈韞	String	是	姓名, 明文传入
idCard	440303200002163115	String	是	身份证号 (18 位, 末位可为 x), 须为校验位合法的有效号码, 明文传入
mobile	13809091009	String	是	手机号 (11 位, 以 1 开头), 明文传入

三项均必填。任一字段缺失或格式非法 (身份证号非 18 位规范号码、手机号非 11 位) 将返回 `head.errorCode = 505062` , 直接拒绝, 不计费。
身份证号仅满足 18 位格式但校验位不正确时, 也可能返回 `505062` (服务处理异常), 请使用真实有效

的身份证号联调。

本接口查询须取得被查询人的合法授权。**授权书不在请求中携带**，由调用方通过商务渠道提交我方，我方进行后续处理（见 1.5）。

3.1.2 请求示例

```
{
  "encryptionType": 1,
  "appKey": "y89snhmd",
  "sign": "<按 2.3 规则计算>",
  "body": {
    "name": "陈韫",
    "idCard": "440303200002163115",
    "mobile": "13809091009"
  }
}
```

3.1.3 响应结构

响应分为 head（网关头部）与 body（业务结果）两部分：

head 字段：

参数	示例	类型	说明
errorCode	0	String	网关返回码。0 = 成功（含查得/查无）；非 0 = 网关级错误，此时无 body
errorMsg	success	String	返回描述
logId	a1b2c3...	String	全链路追踪 ID，排障/对账时需提供
time	103	Number	服务处理耗时（毫秒）
timestamp	1718456789012	Number	响应时间戳（毫秒）

body 字段（仅 head.errorCode = 0 时返回）：

参数	示例	类型	说明
code	001	String	业务结果码。001 = 查得数据【计费】；999 = 查无结果【计费】
msg	成功	String	业务描述
reqid	1kf9x2...	String	本次请求流水号
uid	lgt17901410753043	String	交易流水号（对账用）
result	{...}	Object	业务内容，仅 code = 001 时存在
result.range	"{...}"	String	黑名单结果的 JSON 字符串（对象），需二次解析，结构见 3.1.5

3.1.4 响应示例

1) 查得数据（计费）

```
{
  "head": { "errorCode": "0", "errorMsg": "success", "logId": "a1b2c3d4", "time": 103,
"timestamp": 1718456789012 },
  "body": {
    "code": "001",
    "msg": "成功",
    "reqid": "dkykagq5m17u3",
    "uid": "lgt17901410753043",
    "result": {
      "range": "{\"black_list\":\"1\",\"black_tag04\":\"0\",\"black_tag05\":\"1\",\"black_tag06\":\"0\",\"black_tag07\":\"0\",\"black_tag08\":\"1\",\"black_tag09\":\"0\",\"black_tag10\":\"0\",\"black_tag11\":\"0\",\"black_tag12\":\"1\"}"
    }
  }
}
```

2) 查无结果（计费）

```
{
  "head": { "errorCode": "0", "errorMsg": "success", "logId": "a1b2c3d5", "time": 96, "timestamp": 1718456789013 },
  "body": {
    "code": "999",
    "msg": "查无结果",
    "reqid": "1kf9x2ac",
    "uid": "lgt17901410759871"
  }
}
```

999 表示本次查询已完成、未查得该主体的相关记录——这是一个有效的查询结论，与 001 同样计费（见第五章）。

3) 网关级错误 (无 body)

```
{
  "head": { "errorCode": "505062", "errorMsg": "数据请求异常", "logId": "a1b2c3d6", "time": 12, "timestamp": 1718456789014 }
}
```

3.1.5 result.range 结构说明

result.range 是黑名单结果序列化后的 JSON 字符串。调用方应先对该字符串做一次 JSON.parse / 反序列化，得到一个 JSON 对象（非数组）。

对象内共 10 个字段，值均为 String 类型的 "1" 或 "0"： "1" = 命中， "0" = 未命中。

字段	说明
black_list	是否命中黑名单（总判定）
black_tag04	疑似短期频繁还款失败
black_tag05	疑似短期频繁借贷
black_tag06	疑似短期新机构频繁借贷
black_tag07	疑似短期还款失败比例高
black_tag08	疑似短期多机构逾期
black_tag09	疑似中期频繁还款失败
black_tag10	疑似短期新机构频繁还款失败
black_tag11	疑似当前逾期较严重
black_tag12	疑似高风险客户

`black_list` 是总判定，其余 9 个 `black_tag*` 是各自独立的风险标签，可同时命中多个；标签编号从 04 起，无 01 ~ 03。

△ **重要：**`black_list` 为 "0" **不代表查无**。只要返回 `body.code = 001`，就说明本次已取得该主体的明确黑名单判定结论——「未命中黑名单」本身就是一个有效结论（属查得，计费）。

请用 `result.range` 里的 `black_list` 判断是否命中，**不要用 `body.code` 判断**：`body.code` 只表示「有没有拿到判定结果」，不表示「是不是黑名单」。

`body.code = 999` 才是查无（未查得该主体的相关记录，同样计费）。

说明：`result.range` 的具体字段以实际返回为准，本服务保证原样透出、不裁剪、不改写字段名。标签集合未来可能扩充（如新增 `black_tag13`），建议调用方按「存在即解析、缺失即忽略」做容错处理，不要假设标签数量恒为 10。

解析示例 (JavaScript)：

```
const data = JSON.parse(resp.body.result.range);

// 总判定：注意用 black_list 判断，而不是用 body.code
if (data.black_list === "1") {
  console.log("命中黑名单");
} else {
  console.log("未命中黑名单（这也是有效结论）");
}

// 遍历命中的风险标签，缺失即忽略
const LABELS = {
  black_tag04: "疑似短期频繁还款失败",
  black_tag05: "疑似短期频繁借贷",
  black_tag06: "疑似短期新机构频繁借贷",
  black_tag07: "疑似短期还款失败比例高",
  black_tag08: "疑似短期多机构逾期",
  black_tag09: "疑似中期频繁还款失败",
  black_tag10: "疑似短期新机构频繁还款失败",
  black_tag11: "疑似当前逾期较严重",
  black_tag12: "疑似高风险客户",
};

const hits = Object.keys(LABELS).filter((k) => data[k] === "1");
console.log("命中标签：", hits.map((k) => LABELS[k]));
```

3.2 计费查询数查询（扩展接口）

查询本账户累计计费查询的次数与累计调用次数，用于客户侧自助监控与对账。不返回额度上限/剩余量（本服务无额度限制）。

项目	内容
路径	GET /v1/openapi/zlx/quotaSNHMD
完整地址	http://aiszcloud.cn:8080/v1/openapi/zlx/quotaSNHMD (或 http://aiszcloud.com.cn:8080/v1/openapi/zlx/quotaSNHMD)
鉴权	同 3.1 主查询 (请求体携带 appKey + sign 信封; body 可为 {}, 此时 sign = MD5(appSecret))

响应示例:

```
{
  "errorCode": "0",
  "errorMsg": "success",
  "status": "ACTIVE",
  "serviceUsed": 1280,
  "totalCalls": 1280
}
```

参数	说明
status	账户状态 (ACTIVE / SUSPENDED 等)
serviceUsed	累计计费查询次数 (code = 001 与 code = 999 均计入)
totalCalls	累计有效查询次数 (含查得与查无; 不含被网关拦截的请求)

说明: 无任何额度上限拦截, 仅做计费次数统计。本服务对查得与查无同等计费, 故 serviceUsed 与 totalCalls 在正常情况下一致; 对账请以 serviceUsed 为准。

3.3 健康检查

项目	内容
路径	GET /healthz
完整地址	http://aiszcloud.cn:8080/healthz (或 http://aiszcloud.com.cn:8080/healthz)
鉴权	无
响应	HTTP 200, 纯文本 ok

四、返回码说明

4.1 网关返回码 head.errorCode

errorCode	含义	典型原因
0	成功	调用成功（业务结果见 body.code）
505001	appKey 异常	缺少或非法 appKey
505004	账户信息不存在	appKey 未匹配到账户
505007	服务尚未开通	账户停用 / 过期 / 未开通
505002	账号信息异常	签名校验失败
505003	产品编号异常	保留
505062	数据请求异常	参数非法 / 服务处理异常 / 超时未决 / 系统错误（默认错误码）

服务侧的账户/参数/系统等异常均归一为网关 505062，不计费，并经异步复查兜底。

4.2 业务结果码 body.code（仅 errorCode = 0 时）

code	含义	是否计费
001	查得数据	计费
999	查无结果	计费

注：本服务按有效查询次数计费。只要本次查询产出了确定结论——无论是查得数据（001，含 black_list 为 "1" 命中或 "0" 未命中）还是确认无相关记录（999）——均计费一次。仅当返回 head.errorCode 非 0（未能产出结论）时不计费。

五、计费说明

body.code = 001（查得数据，含「未命中黑名单」这一结论）与 body.code = 999（查无结果）均计入计费查询次数并对客户计费。本服务按有效查询次数计费：查无结果同样消耗一次查询并产出确定结论。网关级错误（head.errorCode 非 0：鉴权失败、参数非法、服务处理异常、系统异常等）一律不计费。计费以最终落库的台账为准，超时未决请求会经异步复查/对账裁定状态，不会重复计费。计费次数可随时通过计费查询数查询（3.2）接口自助核对。

六、使用手册（接入与最佳实践）

6.1 接入流程

1. 向商务申请账户，获取 appKey、appSecret；服务地址见 1.2 接入须知。

2. 与商务确认被查询人授权书的提交方式，并按约定通过商务渠道提交授权书（见 1.5）；授权书不通过接口传输。
3. 按第二章实现加签，先在测试环境联调，再切正式环境。
4. 上线后通过计费查询数查询接口（3.2）监控调用量与计费次数。

6.2 幂等与重试

- 客户端建议为每笔查询设置合理超时（≥ 10 秒）。
- 收到网络超时/无响应时可安全重试：本服务基于内部流水号做幂等，不会因重试重复计费。
- 请勿对已明确返回 `head.errorCode` 的请求做无差别重试（如参数错误 505062、鉴权错误 505001 / 505002 / 505004），应先修正再发起。

6.3 错误处理建议

现象	排查方向
505001 / 505004	检查 appKey 是否正确、是否用错环境账户
505002	检查签名算法（排序/空值剔除/UTF-8/小写 hex）；确认实际发送的每一个 body 字段都参与了加签
505007	联系商务确认账户状态与有效期
505062	检查 name、idCard、mobile 是否三项齐全，身份证号是否有效（含校验位）、手机号是否为 11 位；若入参正常仍持续出现，记录 logId 联系我方
标签全为 "0"	属正常结论（该主体未命中任何风险标签），不是异常、不是查无，该次调用计费
code=999 持续出现	确认三要素是否为同一人的真实有效信息；该结果表示未查得相关记录，属有效结论，同样计费

任何异常排查请一并提供响应中的 `head.logId`，便于我方全链路定位。

6.4 联调自检清单

- ☐ 服务地址（`aiszcloud.cn:8080` 或 `aiszcloud.com.cn:8080`）、appKey、appSecret、环境匹配无误
- ☐ 已与商务确认授权书提交方式，并已通过商务渠道提交被查询人授权书（请求中不携带授权书）
- ☐ 待签名串严格按 ASCII 升序拼接、剔除空值、UTF-8、MD5 小写
- ☐ name、idCard、mobile 三项均以明文传入且都不为空
- ☐ 使用校验位合法的真实身份证号（勿用随意编造的 18 位号码），手机号为 11 位真实号码
- ☐ 若额外传了本接口不需要的字段，已把它们计入加签范围
- ☐ 能正确解析 `head.errorCode` 与 `body.code` 两级状态
- ☐ 已知悉 999 查无结果同样计费，业务侧计费预估按有效查询次数而非查得数测算
- ☐ 已实现对 `result.range` 字符串的二次解析（结果为 JSON 对象）与字段容错
- ☐ 是否命中黑名单取自 `range` 里的 `black_list`，未用 `body.code` 代替判断
- ☐ 已理解「标签全 0 也是有效结论且计费」，风控策略不因全 0 而重复查询

- ☐ 已按「存在即解析、缺失即忽略」处理标签字段（预留标签扩充）
- ☐ 已实现超时重试（依赖幂等，不重复计费）

附录：术语表

术语	说明
appKey	公开账户标识，随请求明文传输
appSecret	加签密钥，仅本地保存用于计算 sign，切勿泄露或随请求传输
logId	全链路追踪 ID (= head.logId) ， 排障/对账唯一凭据
uid	交易流水号 (= body.uid) ， 对账用
range	黑名单结果的 JSON 字符串，需二次解析（结构见 3.1.5）
black_list	黑名单总判定字段， "1" 命中 / "0" 未命中
black_tag*	独立风险标签字段 (black_tag04 ~ black_tag12)， "1" 命中 / "0" 未命中
有效查询	产出了确定结论的查询 (body.code = 001 或 999) ， 即本服务的计费单位
授权书	被查询人签署的个人信息查询授权文件；不通过接口传输，由调用方经商务渠道提交我方处理